

Presentation Script

Bitcoin Volatility Prediction — Capstone Talk (15–20 minutes)

Mohammed Hejairi

How to Use This Script

This is a slide-by-slide speaking script for the 17-slide deck (`capstone-btc-volatility.pptx`), timed to fill a 15–20 minute talk at a natural, unhurried pace (roughly 130–140 words per minute). Each section below is headed with its slide number, title, and a target time — both **per-slide** and **cumulative**, so you can check your pace against a clock while you rehearse.

The text is written the way you’d actually say it out loud, not the way you’d write it — short sentences, contractions, and natural pauses. Treat it as a guide, not a script to memorize word-for-word: know the beats of each slide (what it’s for, what number matters, what the takeaway is) and let the phrasing come out naturally when you present.

[pause] marks a spot to stop, breathe, and let a number or point land before moving on — these matter more than they look like on paper.

Total target time: ~18–19 minutes, leaving a comfortable buffer inside a 20-minute slot for Q&A or a slower pace on the day.

Slide 1 — Title

Target: 0:30 (cumulative 0:30)

Good [morning/afternoon], everyone. My name is Mohammed Hejairi, and today I’m presenting my capstone project: **Bitcoin Volatility Prediction**.

[pause]

The one-line version of this whole talk is on the slide: I’m not forecasting where Bitcoin’s price is going. I’m forecasting how much it’s going to move — up or down, I don’t care which. That distinction is the whole project, and I’ll come back to it in a second.

Slide 2 — Introduction

Target: 1:00 (cumulative 1:30)

Let me start with what this project actually is, in four points.

First — Bitcoin doesn't move randomly and evenly over time. It swings hard, and it swings in *bursts*. You get long calm stretches, and then a sharp spike, and then calm again. That pattern has a name in finance: **volatility clustering**. It's real, it's well documented, and — this is the important part — it means volatility itself has structure you can learn from, even if price direction doesn't.

Second, and this is the core design decision of the whole project: most crypto projects out there try to predict price direction. I deliberately didn't. Predicting *direction* in a market this efficient is close to a coin flip. Predicting *turbulence* — how much things will move — is a genuinely tractable machine learning problem, and it's also the number that actually matters to traders and risk desks.

Third, the target I'm predicting is **7-day realized volatility, 168 hours ahead**. Concretely: the rolling standard deviation of hourly returns over a 168-hour window, and I'm forecasting that value a full 168 hours — a week — into the future.

Fourth, the data: hourly BTC/USDT candles from Binance, pulled through the CCXT library, covering 2017 through 2026. [pause]

Slide 3 — Problem Statement and Aim

Target: 1:15 (cumulative 2:45)

So why does this matter? The raw problem is this: Bitcoin's price swings unpredictably at short timescales, and traders, exchanges, and risk desks have no reliable way to anticipate how turbulent the *next* stretch of trading is going to be. That leads to mispriced risk, thin margin buffers, and position sizes that are wrong for the conditions.

I turned that into a structured, SMART objective: predict Bitcoin's 7-day realized volatility, 168 hours ahead, using lagged price and volume history plus technical indicators — all sourced hourly through CCXT. The output is a continuous, hourly-updating volatility estimate that a trader or risk desk can plug directly into position sizing or margin decisions. And I benchmarked it against two baselines: naive persistence, and a seasonality-aware Holt-Winters model.

One thing worth flagging explicitly: this is a **regression problem**, not classification. I'm not predicting "high volatility" versus "low volatility" as a category. The downstream uses — Black-Scholes pricing, Value-at-Risk, position sizing — all need an actual number, not a bucket.

Slide 4 — Data Summary

Target: 1:00 (cumulative 3:45)

Quickly, what the data actually looks like. Asset is BTC/USDT, hourly candles, over eight-plus years through the start of 2026, with a 168-hour target window — that's the 7 days.

Each record is one hour of trading: open, high, low, close, volume — plus a set of engineered features on top: returns, rolling volatility, EMA, RSI, ATR, Bollinger band width, MACD. Every one of those features is lagged one to three hours, so the model only ever sees the past when it makes a prediction — no leakage. And the target itself is `volatility_7d`, shifted 168 hours into the future relative to the features.

One design choice I want to highlight: reproducibility. I fetch the data **once**, save it to a CSV, and every notebook cell after that reads from the CSV — never the live API again. A second, separate CSV accumulates genuinely new candles specifically for live testing later on, which becomes important in a few slides.

Slide 5 — EDA: Key Findings

Target: 1:15 (cumulative 5:00)

Three findings came out of exploratory analysis that directly shaped the modeling.

One — the target is right-skewed with fat tails, exactly what you'd expect from volatility clustering: long calm stretches, sharp spikes. And the autocorrelation function decays *slowly* across many lags rather than dropping to zero quickly. That's confirmation the autocorrelation is real and exploitable — it's not noise.

Two — range-based measures like ATR and Bollinger width correlate strongly with the target. That matters because those are built from intraday high-low ranges, not from close-to-close returns, so it's genuinely independent signal — not just a restatement of the volatility target in disguise.

Three, and this one's a negative result worth stating plainly: I tested whether volatility varies significantly by time of day. Visually, in the boxplots, it looks like it might. But a chi-square test found **no significant association** — p is approximately 1.0. And that gets corroborated independently: when I fit Holt-Winters, its own seasonal parameter converges to essentially zero on its own. Two different methods agreeing that time-of-day isn't a real signal here.

Slide 6 — Modeling Approach

Target: 1:15 (cumulative 6:15)

Before I get into the individual models, one deliberate design decision: I excluded Linear Regression, Random Forest, and XGBoost from the core comparison. Not because they can't be used with time series — they can, with lag features — but because they have **no internal notion of time**. Every row is just an unordered bag of numbers to them; time only enters through hand-engineered lag columns. Every model I *did* use has time built into its own mathematics.

On preprocessing: everything is lagged one to three hours with no lookahead, zero-volume candles are guarded before rolling calculations so they don't produce infinities, and no feature scaling is needed — every model fits directly on returns or already-comparable regressors. I also tested a log transform for the right-skewed target, which I'll come back to.

On validation: a strict chronological 80/20 split — never random k-fold — `TimeSeriesSplit` for hyperparameter tuning, naive-persistence and Holt-Winters as baselines, and — this is the part I'm most proud of methodologically — live testing on genuinely unseen future data, which I'll show results for later.

Three model families: GARCH, moving to EGARCH and GJR-GARCH; HAR-RV, moving to HAR-RV-X; and EWMA. Each one has a base version and an improved version, so you can see exactly what the improvement bought.

Slide 7 — Model 1: GARCH Family

Target: 1:15 (cumulative 7:30)

The base version is GARCH(1,1) — today's variance equals a baseline, plus a reaction to yesterday's shock, plus persistence from yesterday's variance. Three parameters, fit by maximum likelihood.

The improvement is **asymmetry**. Plain GARCH treats a 5% drop and a 5% rise identically, because the shock term enters squared. EGARCH and GJR-GARCH add a leverage term so that bad news raises volatility more than good news does — which matches how markets actually behave. Note this is an equation change, not just a wider hyperparameter search.

I selected among four specifications — EGARCH and GJR, each with normal or Student-t errors — using AIC, which is the right criterion here since these are maximum-likelihood models without a validation loss in the usual sense. The winner: GJR-GARCH with Student-t errors, AIC of about 96,565 versus 115,563 for basic GARCH.

[pause] And the numbers back it up — RMSE drops from 0.0073 to 0.0039, R^2 goes from about -20 to about -5. Still negative, still weak — and I'll be upfront about why: GARCH's forecast mean-reverts toward a long-run average variance well before 168 hours are up. That's a documented property of GARCH at long horizons, not a defect in my implementation.

Slide 8 — Model 2: HAR-RV / HAR-RV-X

Target: 1:30 (cumulative 9:00)

This is the best model in the whole project, so I want to spend a bit more time on it.

The base version, HAR-RV, regresses volatility on volatility measured over three horizons: the past day, the past week, the past month — 24, 168, and 720 hours. The idea, from Corsi's

original 2009 paper, is that it captures **long memory**: different types of market participants — day traders, weekly rebalancers, long-term holders — act on different timescales, and this model lets each one contribute.

The improvement is the “X” — adding ATR as a *percentage* of price, not raw dollars. And there’s a real story here worth telling: the first version I built used raw-dollar ATR, and it was a **~45,000x scale mismatch** against every other regressor. That bug actually made HAR-RV-X score *worse* than plain HAR-RV, until I caught it by treating that bad result as suspicious rather than just accepting it. Fixing the scale took R^2 from -0.65 to +0.18 — the single biggest win in the entire project.

On top of that, I added Ridge regularization, tuned by GridSearchCV over TimeSeriesSplit, because all four regressors are measuring volatility in different ways and are therefore correlated — unpenalized coefficients would be unstable.

[pause] Base HAR-RV: R^2 of 0.071. HAR-RV-X: R^2 of 0.184. And later robustness checks — which I’ll touch on in the conclusion — added two more regressors and a log-target transform, pushing that further to **0.2419 historical, 0.1779 on live data**. This is the model that wins on both the historical split and — more importantly — on genuinely unseen live data.

Slide 9 — Model 3: EWMA (RiskMetrics)

Target: 1:00 (cumulative 10:00)

Third model: EWMA, the RiskMetrics approach from J.P. Morgan, 1996. The base version uses λ equals 0.94 — the published industry-standard decay factor, used exactly as-is, zero fitting to this dataset. The formula is simple: next variance is λ times today’s variance, plus one-minus- λ times today’s squared return.

The improvement is backtesting λ instead of trusting the textbook value — a grid search from 0.85 to 0.995, widened after 0.99 won at the original grid’s edge, which was a signal the true optimum sat higher. Scored with TimeSeriesSplit, on training data only.

Quick aside on why this is here instead of an LSTM: I actually had an LSTM in this slot originally, and I dropped it. It carried no financial theory of its own — it was a black box competing against models that risk desks can actually reason about. EWMA is the exact estimator those desks already trust, with one transparent, interpretable parameter.

[pause] Basic EWMA: R^2 of -0.55. Tuned, at λ equals 0.995: R^2 of 0.059. A real improvement, and this time the optimum landed inside the grid, not at its edge.

Slide 10 — Model Comparison

Target: 1:00 (cumulative 11:00)

Putting it all together, scored on the same held-out data. Naive persistence and Holt-Winters as baselines — Holt-Winters actually does worse than doing nothing, R^2 of -2.24. GARCH, even in its improved form, also loses to naive persistence. HAR-RV-X and EWMA both beat it.

[pause] The winner is HAR-RV-X — with the later-added regressors and log-target transform — beating naive persistence by **17.1% lower RMSE**. That’s not a marginal artifact; it’s a genuine, working model. EWMA is a solid second place and also clears the persistence bar.

But historical splits can lie to you — through subtle leakage, or just because you iterated against that same test set while building the model. So the next section is the one that actually matters more.

Slide 11 — Live Testing on Unseen Data

Target: 1:15 (cumulative 12:15)

This is the part of the project I’d point to if someone asked “why should I trust this.” I didn’t just stop at the historical hold-out — I built a second, completely separate validation.

Step one: fetch a live-test period — January 2nd 2026 through yesterday — that was **never touched during training**, which stops at the start of 2026. **Step two:** score every hour in that window with the already-saved GARCH, HAR-RV-X, and EWMA models — no retraining, no peeking. **Step three:** a 168-hour-ahead prediction can’t be checked until 168 real hours have actually passed, so I only score what’s already elapsed — 5,496 of 5,664 predictions so far, with the rest resolving as more days go by. **Step four:** compare that to the historical ranking.

[pause] And the result: HAR-RV-X — the historical winner — wins on live data too, on every metric. That’s the difference between a model that looks good and a model that’s actually generalizing. A historical split can flatter you. A window the model has genuinely never seen can’t.

Slide 12 — Live Test Results

Target: 1:00 (cumulative 13:15)

The numbers: HAR-RV-X on live data — R^2 of **0.1779**, the best of every model tested, on every metric. EWMA is a close second, and actually posts the lowest MAPE of anything here, at 22.9%. Naive persistence, without an easy horizon propping it up anymore, drops to a *negative* R^2 .

[pause] And GARCH collapses entirely — R^2 of **-110.7**, MAPE over 400%. I want to be clear that this isn’t a bug in my implementation — it’s the same structural weakness I flagged on the GARCH slide, mean-reversion toward a long-run average well before 168 hours are up. It just shows up much more starkly once you take away the flattering effect of a historical split.

Slide 13 — Resolution Sensitivity

Target: 1:00 (cumulative 14:15)

One extra question I asked myself: does the sampling frequency change how good these models can get? I built a companion pipeline at 5-minute resolution instead of hourly, for both HAR-RV-X and EWMA.

Quick note before the numbers: RMSE isn't directly comparable across resolutions, because the target's absolute scale changes by construction — so R^2 is the fair comparison here.

[pause] Both models improve at 5-minute resolution. HAR-RV-X goes from 0.178 to 0.240 — about 1.3x. EWMA improves far more dramatically, from 0.013 to 0.147 — roughly **11x**. That's a real finding: EWMA in particular seems to benefit a lot from finer-grained data, more than HAR-RV-X does. This was one of eighteen robustness checks I ran on the project — I'll summarize those next.

Slide 14 — Conclusion

Target: 1:30 (cumulative 15:45)

Let me pull this together into implications, limitations, and next steps.

Business implications are concrete: this continuous volatility number feeds directly into Black-Scholes option pricing, drives Value-at-Risk and capital reserve calculations, enables volatility-scaled position sizing, and informs exchange margin and liquidation thresholds. I'll actually walk through two of these as worked examples in the next two slides.

Limitations, stated honestly: volatility is partly driven by news that simply isn't present in price data — that's a hard ceiling on how accurate any price-only model can get. GARCH's mean-reversion at long horizons is a structural weakness, not something I can tune away. I ran **eighteen robustness checks** across the project — testing things like XGBoost, LSTM, stacked ensembles, ETH spillover signals, winsorizing, alternative feature sets — and only **four survived validation** and were adopted; fourteen didn't help, and I documented why for each one rather than just discarding them silently. And this project uses a single asset — no cross-asset spillover signal from other coins.

Future improvements: LLM-scored news sentiment as an added feature, to attack that news-driven ceiling directly. Rolling or expanding-window GARCH refits. Automating live testing on a schedule to build a longer track record. And on-chain metrics — exchange netflows, active addresses — which would be a genuinely independent information source, distinct from every price-history feature I've tested so far.

Slide 15 — Practical Application: Sizing a Trade

Target: 1:15 (cumulative 17:00)

I want to make this concrete with two real worked examples, using the model's actual live forecast — not hypothetical numbers.

Imagine a hedge fund risk desk using this. On August 25th, 2026, the model's real, live forecast was that Bitcoin would swing about **6% over the coming week**. If that desk is willing to risk \$50,000 on a move of that size, the math points to a position worth about **\$823,000** — the bigger the forecasted move, the smaller the position has to be to keep the same dollar risk. From there, that position carries roughly a **5% chance of losing more than about \$82,000** over the week — that's the number that actually goes into a risk report. And you can compare that forecast to how the options market is pricing the same risk: if options are pricing in a much bigger move than the model expects, they may be overpriced relative to what the model thinks is fair.

[pause] I want to be upfront about what this is and isn't: the model explains about 18% of real volatility swings, with typical errors around a quarter of the forecast. That's useful — genuinely useful — but it's one input alongside market context and judgment, not a replacement for either. And it's worth remembering GARCH performed far worse over this exact horizon, so this is a real improvement, not a finished, closed system.

Slide 16 — Practical Application: Pricing an Option

Target: 1:15 (cumulative 18:15)

Second example: an options trader. Options pricing — Black-Scholes — needs a volatility number as an input, and that's exactly what this model produces.

Scale that same short-term forecast up to the yearly figure options pricing conventionally uses, and it comes out to about **44% annualized volatility**. Price a hypothetical 7-day Bitcoin option using that number, and the model's own fair-value estimate is about **\$1,177**. Now compare that to how the market is actually pricing it: if the market is using something like 50% volatility instead, that same option comes out around **\$1,454** — about \$280 more expensive than the model thinks is fair. That gap is the interesting part: something priced *more* expensively than the model expects might be worth selling; something priced cheaper — say, a 35% assumption, around \$792 — might be worth buying. Always alongside the market's own price, never in place of it.

[pause] Same caveat as before, and it's an important one: this never overrides a liquid market's own quoted price. It's most useful when no market price exists at all, or for spotting relative mispricing — and even then, any real trader would size it carefully and combine it with other information, not trade on it alone.

Slide 17 — Thank You

Target: 0:20 (cumulative 18:35)

That's the project: a volatility forecasting model that beats naive persistence and a seasonal baseline, validated not just historically but on genuinely unseen live data, with two concrete use cases in risk management and options pricing.

Thank you — I'm happy to take any questions.

Pacing Checkpoints

Use these as quick glances at a clock while you rehearse — if you're meaningfully ahead or behind one of these, adjust pace on the next section rather than trying to catch up all at once.

Checkpoint	Slide	Target elapsed time
Intro + problem framed	End of Slide 3	~2:45
Data + EDA covered	End of Slide 5	~5:00
All three models presented	End of Slide 9	~10:00
Comparison + live testing done	End of Slide 12	~13:15
Conclusion reached	End of Slide 14	~15:45
Both worked examples done	End of Slide 16	~18:15
Finished	Slide 17	~18:35